

Chado Schema Deep Dive

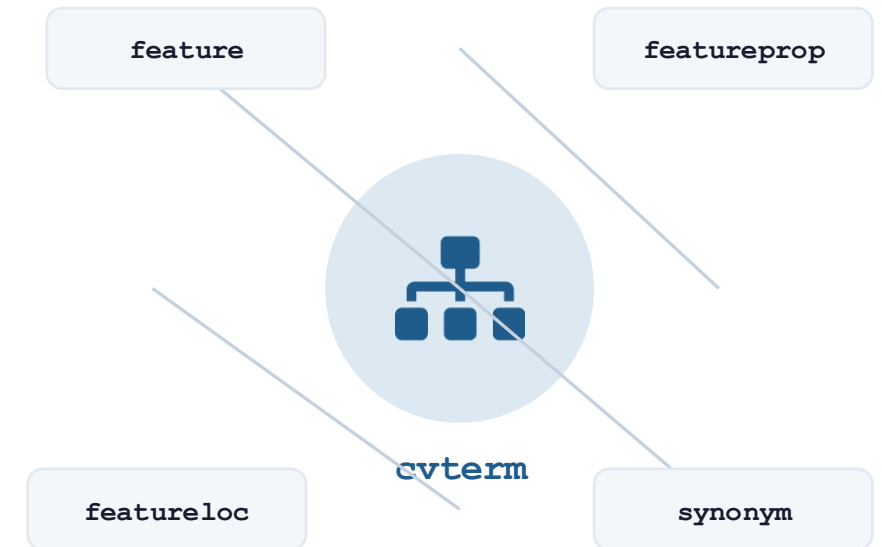
& Ontology Implementation

How the schema is wired onto the cvterm bridge

1k1 Rice Genome Project

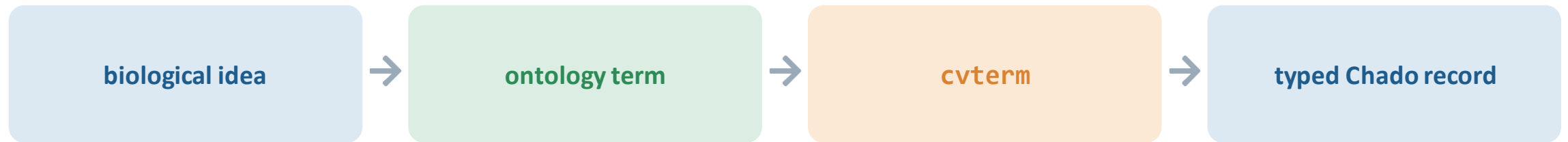
Continues from “Ontology Concepts for Chado” (J. Detras)

June 3, 2026



From the cvterm bridge to the whole schema

Jeffrey covered WHY cvterm is the semantic bridge. This talk covers HOW the schema hangs off it.



His talk: the vocabulary — cv, cvterm, dbxref, relationships.

This talk: the plumbing — the actual tables you load into and query for SNP-Seek.

type_id → cvterm, repeated everywhere

Chado has very few “data” tables — a handful of TYPED tables that get meaning from cvterm.

`feature.type_id`



cvterm

is this a gene? a SNP? a chromosome?

`featureprop.type_id`



cvterm

what kind of property is this annotation?

`relationship tables`



cvterm

is_a, part_of, derives_from

`synonym.type_id`



cvterm

which naming system this name belongs to

New feature type tomorrow? Add a cvterm, not a table. The schema doesn't change — that's the whole design philosophy.

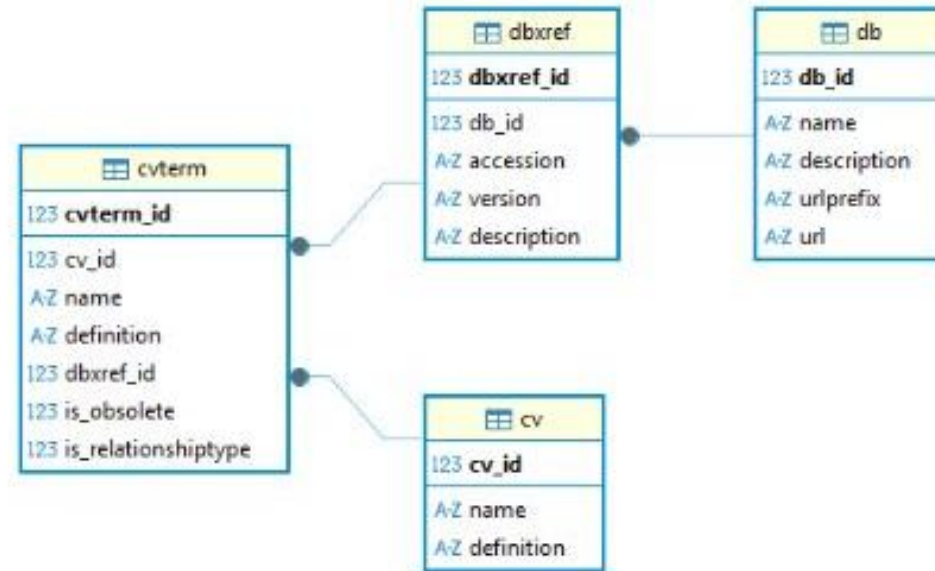
Commonly used Chado tables

The handful of tables you'll actually load into and query for SNP-Seek.

db	<i>external databases / sources (GenBank, Ensembl)</i>	snp_featureloc	<i>genomic location of each SNP feature</i>
cv	<i>controlled vocabularies (SO, GO)</i>	stock	<i>biological samples / accessions</i>
cvterm	<i>ontology terms — “SNP”, “chromosome”, “gene”</i>	stock_sample	<i>links stock to datasets / sequencing runs</i>
organism	<i>taxonomy for species & subspecies</i>	variantset	<i>a group of variants for a study / dataset</i>
feature	<i>central table — genes, SNPs, contigs</i>	variant_variantset	<i>join table: variants → their variant sets</i>
snp_feature	<i>SNP-specific features (variant loci)</i>		

Ontology-driven

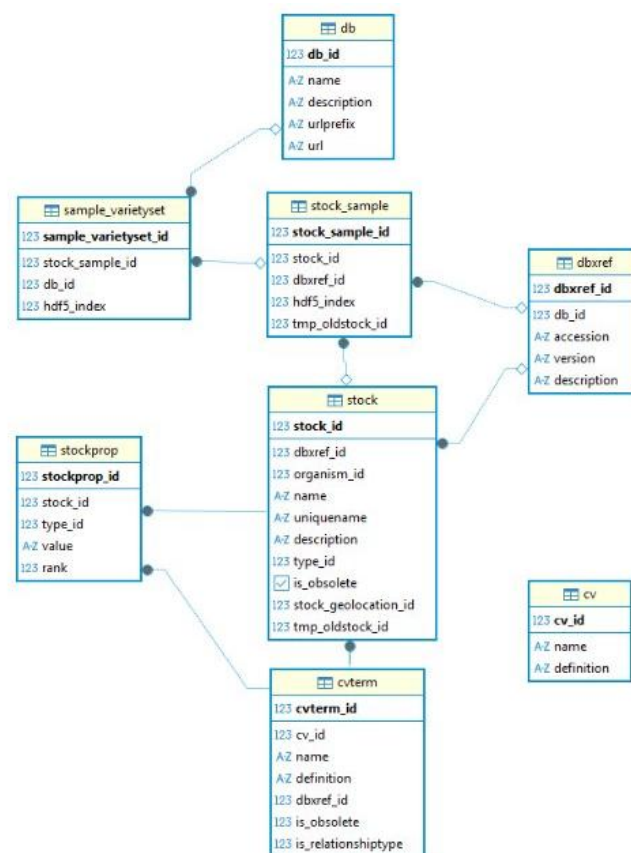
Every biological type — SNP, gene, transcript — is defined through cvterm.



cvterm ↔ dbxref ↔ db, with cv defining the vocabulary each term belongs to.

Modular, linked by foreign keys

Each concept — organism, feature, stock, variant — is stored separately but linked.



The feature table

Genes, SNPs, contigs, chromosomes — all rows in ONE table. type_id is what differs.

`feature_id`

primary key

`uniquename`

stable unique identifier

`name`

human-readable label

`organism_id → organism`

which rice line / reference

`type_id → cvterm`

WHAT this feature is



Jeffrey's example, landed

```
feature.type_id  
↓  
cvterm.cvterm_id  
↓  
SO:0000704 (gene)
```

featureloc and featureprop

A feature alone doesn't say where it is, or carry its notes — these two tables do.



featureloc

what chromosome, what position, what strand

- **srcfeature_id**
 - the feature it sits on (chromosome / contig)
- **fmin, fmax**
 - start / end coordinates
- **strand**



featureprop

notes, descriptions, scores

Anything you attach to a feature

Also has type_id → cvterm

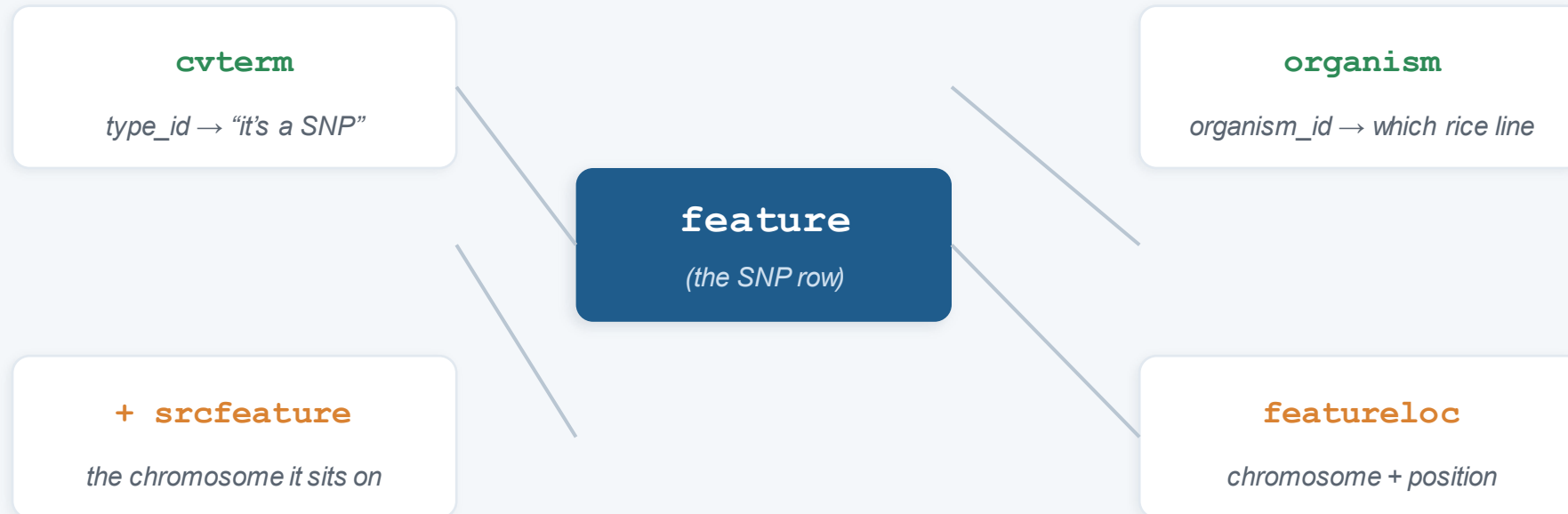
The property TYPE is itself a controlled term — the same pattern again.



Loading gotcha: *fmin is zero-based and interbase — not the 1-based number in a genome browser. Classic off-by-one bug.*

Trace one SNP through the schema

Four tables fully describe a single SNP: what it is, whose genome, exactly where.



Stocks, variant sets — and what is NOT in PostgreSQL

The metadata lives in Chado. The genotype matrix does not.



organism

reference genomes & rice lines



stock / stock_sample

the 3,000+ accessions & their samples



variantset / variant_variantset

named groupings of variants (BrAPI)

THE BOUNDARY

PostgreSQL/ Chado

WHAT and WHERE — features, locations, accessions, set definitions

HDF5 files

the dense matrix — ~60 billion data points —
NOT in PostgreSQL

Synonyms: one locus, many names

Don't build a wide locus_mapping table — use synonym + feature_synonym, typed by cvterm.

The naming systems:

- MSU7 / LOC_ (LOC_Os01g01307)
- IRIC
- RAP representative / RAP predicted
- FGENESH
- Per-genome: IR64, MH63, N22, Azucena

***New naming system = add a cvterm, not a column.
Same move as everything else.***

synonym

the alternative name + a type_id (cvterm) for which naming system

feature_synonym

links the synonym to the feature it names

```
SELECT ... WHERE feature.name = ? UNION SELECT ...  
JOIN feature_synonym JOIN synonym WHERE synonym.name =  
?
```

A synonym-aware lookup is a UNION — recognize the shape, don't memorize the SQL.

Three pillars

1



Ontology-driven typing

Meaning lives in cvterm, referenced by type_id everywhere.

2



Extensibility

New types and new names are new cvterms — not schema changes.

3



Integrity

Foreign keys hold features, locations, stocks, and sets together.

Most loader failures are term-resolution problems — a name in the file that doesn't match a term. Now you know which tables they land in.



Questions?

What would you like to trace through the schema together?

Where is a gene's description stored? → *featureprop*

Every SNP on chr 1? → *featureloc* via *srcfeature*
genotype call in PostgreSQL? → *HDF5* *boundary*

Why not the